ChatGPT said:

Of course. Let's go through the entire Python code step by step, with particular attention to how each part relates to the finite element method (FEM).

The objective of the code is to solve a system of equations using the **partitioning method**, apply displacement boundary conditions, calculate the unknown displacements, and finally calculate the reaction forces.

1. Import NumPy

Python

```python
import numpy as np
```

NumPy is a Python library for numerical calculations. We use it for:

- Creating matrices and vectors.
- Performing matrix multiplication.
- Extracting submatrices.
- Solving systems of linear equations.

For example:

Python

```python
A = np.array([[2, 1], [1, 3]])
```

creates a 2 × 2 matrix.

2. Define the global stiffness matrix

Python

```python
K = np.array([
    [ 10, -10,   0],
    [-10,  20, -10],
    [  0, -10,  10]
], dtype=float)
```

This is the global stiffness matrix obtained after assembling the element stiffness matrices.

We have three degrees of freedom, corresponding to three nodes.

$$\mathbf{K} = \begin{bmatrix} 10 & -10 & 0 \\ -10 & 20 & -10 \\ 0 & -10 & 10 \end{bmatrix}$$

The parameter `dtype=float` ensures that the matrix elements are floating-point numbers rather than integers.

3. Define the global force vector

Python

```
F = np.array([0, 100, 0], dtype=float)
```

This represents the externally applied nodal forces:

$$\mathbf{F} = \begin{bmatrix} 0 \\ 100 \\ 0 \end{bmatrix}$$

We have applied a force of 100 at node 1.

Notice that the force at node 0 is zero in the applied force vector. However, the support reaction at node 0 is not necessarily zero.

4. Define the constrained degrees of freedom

Python

```
constrained = [0]
```

This is particularly important.

Python uses zero-based indexing, meaning:

Python index	FEM node
0	Node 0
1	Node 1
2	Node 2

Therefore:

```python
constrained = [0]
```

means that displacement at node 0 is prescribed.

$$u_0 = 0$$

If you wanted to prescribe displacements at nodes 0 and 2, you would write:

```python
constrained = [0, 2]
```

5. Identify the free degrees of freedom

```python
all_dofs = np.arange(len(F))
free = np.setdiff1d(all_dofs, constrained)
```

Let's examine these two commands.

First command:

Python

```
all_dofs = np.arange(len(F))
```

`len(F)` returns the number of elements in the force vector.

Here:

$$\text{len}(F) = 3$$

`np.arange(3)` generates:

Python

```
[0, 1, 2]
```

Thus, `all_dofs` contains all degrees of freedom.

Second command:

Python

```
free = np.setdiff1d(all_dofs, constrained)
```

This identifies the degrees of freedom that are not constrained.

Since:

Python

```
all_dofs = [0, 1, 2]
constrained = [0]
```

we obtain:

Python

```
free = [1, 2]
```

Therefore:

$$u_f = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

This is an automatic way of identifying the free degrees of freedom.

6. Define the displacement vector

Python

```
u = np.zeros(len(F))
u[constrained] = 0
```

The first command creates a displacement vector containing zeros:

$$\mathbf{u} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

The second command assigns zero displacement to the constrained degrees of freedom.

In this particular example, the second command is actually redundant because we have already initialized the entire displacement vector with zeros.

However, it becomes important when we have nonzero prescribed displacements.

For example:

Python

```
u[constrained] = 0.005
```

would prescribe a displacement of 0.005 at the constrained degrees of freedom.

7. Partition the global stiffness matrix

This is the central part of the code.

Python

```
Kff = K[np.ix_(free, free)]
Kfc = K[np.ix_(free, constrained)]
```

Here, `np.ix_()` allows us to extract specific rows and columns from a matrix.

We have:

$$\mathbf{K} = \begin{bmatrix} 10 & -10 & 0 \\ -10 & 20 & -10 \\ 0 & -10 & 10 \end{bmatrix}$$

The free degrees of freedom are 1 and 2, while the constrained degree of freedom is 0.

Therefore:

$$K_{ff} = \begin{bmatrix} 20 & -10 \\ -10 & 10 \end{bmatrix}$$

And:

$$K_{fc} = \begin{bmatrix} -10 \\ 0 \end{bmatrix}$$

These are exactly the matrices required for the partitioning method.

8. Extract the corresponding force vector

Python

```
Ff = F[free]
```

This extracts the forces corresponding to the free degrees of freedom.

Since:

$$\mathbf{F} = \begin{bmatrix} 0 \\ 100 \\ 0 \end{bmatrix}$$

we obtain:

$$F_f = \begin{bmatrix} 100 \\ 0 \end{bmatrix}$$

Notice that we only extract forces associated with free degrees of freedom when solving for unknown displacements.

9. Solve the reduced system

Python

```
uf = np.linalg.solve(
    Kff,
    Ff - Kfc @ u[constrained]
)
```

This command solves the reduced equilibrium equation:

$$K_{ff} u_f = F_f - K_{fc} u_c$$

Here, `@` represents matrix multiplication.

In our example, the prescribed displacement is zero:

$$u_c = 0$$

Therefore:

$$\begin{bmatrix} 20 & -10 \\ -10 & 10 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 100 \\ 0 \end{bmatrix}$$

Solving this system gives:

$$u_f = \begin{bmatrix} 10 \\ 10 \end{bmatrix}$$

A note about `np.linalg.solve`:

This function solves a system of linear equations:

$$Ax = b$$

It does not explicitly calculate the inverse of the matrix. Numerically, this is generally preferable to calculating the inverse.

10. Reconstruct the complete displacement vector

Python

```
u[free] = uf
```

Previously, we had:

$$\mathbf{u} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Now we insert the calculated free displacements:

$$\boxed{\mathbf{u} = \begin{bmatrix} 0 \\ 10 \\ 10 \end{bmatrix}}$$

This is the complete displacement vector.

11. Calculate reaction forces

Python

```
R = K @ u - F
```

We return to the original global equilibrium equation:

$$Ku = F + R$$

Therefore:

$$\boxed{R = Ku - F}$$

In our example:

$$Ku = \begin{bmatrix} -100 \\ 100 \\ 0 \end{bmatrix}$$

Consequently:

$$R = \begin{bmatrix} -100 \\ 100 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 100 \\ 0 \end{bmatrix}$$

Thus:

$$\boxed{R = \begin{bmatrix} -100 \\ 0 \\ 0 \end{bmatrix}}$$

The support at node 0 provides a reaction force of -100.

The reaction forces at the free degrees of freedom should be zero (up to numerical precision).

---

12. Summary of the complete FEM workflow

The code follows this sequence:

Step	Python command	FEM operation
1	`K = np.array(...)`	Define global stiffness matrix
2	`F = np.array(...)`	Define global force vector
3	`constrained = [0]`	Define boundary conditions
4	`np.arange()`	Identify all DOFs

Step	Python command	FEM operation
5	`np.setdiff1d()`	Identify free DOFs
6	`np.zeros()`	Initialize displacement vector
7	`np.ix_()`	Extract partitioned stiffness matrices
8	`F[free]`	Extract free force vector
9	`np.linalg.solve()`	Solve reduced equilibrium equations
10	`u[free] = uf`	Reconstruct displacement vector
11	`K @ u - F`	Calculate support reactions

One suggestion for developing your FE code

I would recommend adding the following command immediately after solving the reduced system:

Python

```
print("\nCheck equilibrium:")
print("K @ u =", K @ u)
print("F + R =", F + R)
```

These two vectors should be identical:

$$Ku = F + R$$

This provides a useful check that your solution satisfies the global equilibrium equations.

The most important concept to understand: We use the reduced stiffness matrix to calculate unknown displacements, but we use the original global stiffness matrix to calculate reaction forces. This distinction is fundamental to implementing the partitioning method in a general-purpose FEM program.